

Geometry Operations with Vector Data

Alexander Abuabara

DAEN 489 ISEN 489

Spring 2026



ENGINEERING
TEXAS A&M UNIVERSITY

Att. Code ####

Overview

So far, we have:

- Explained the **structure of geographic** datasets
(core components: geometry, attributes, topology)
- **Manipulate** geographic datasets based on their:
 - **Non-geographic attributes** (filters and subsets), and
 - And **spatial relations** (subsets and topological relations)

This week focuses on manipulating even more the geographic elements of spatial objects. By:

- Simplifying,
- Creating buffers, and
- Converting vector geometries.

Learning Outcomes (Week 5/Assignment #3)

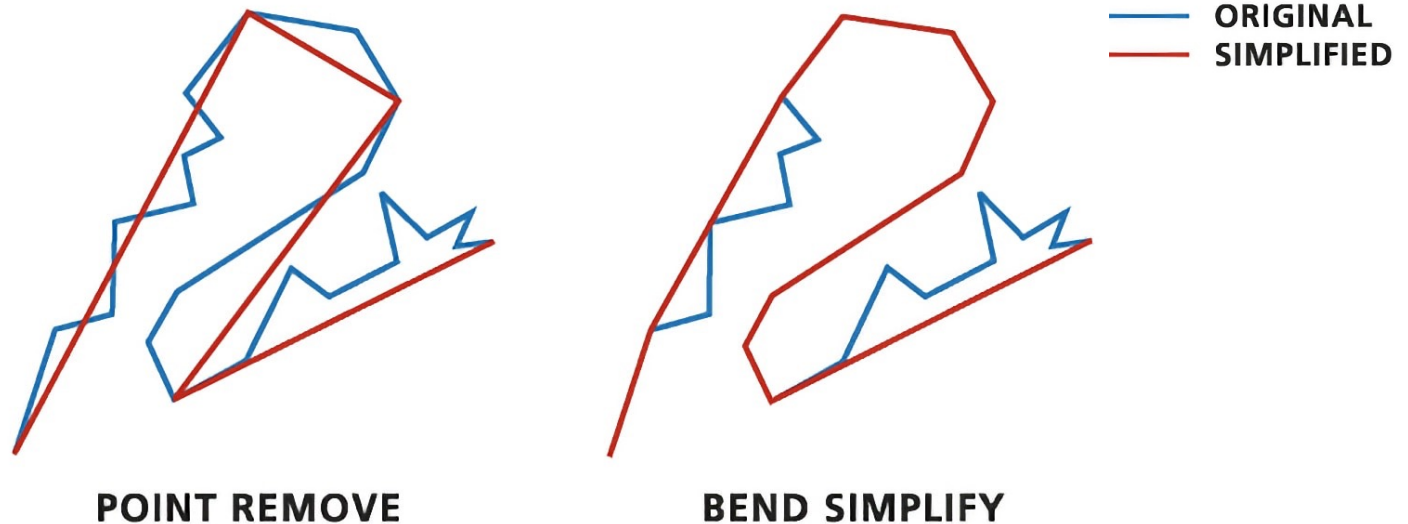
By completing this week/assignment, you should be able to:

1. Design a spatial data workflow from raw public datasets
2. Perform spatial transformations and joins
3. Evaluate methodological choices in spatial overlay analysis
4. Integrate multiple administrative boundary datasets
5. Produce summary statistics from engineered spatial data
6. Clearly communicate your analytical plan and tradeoffs

Emphasis

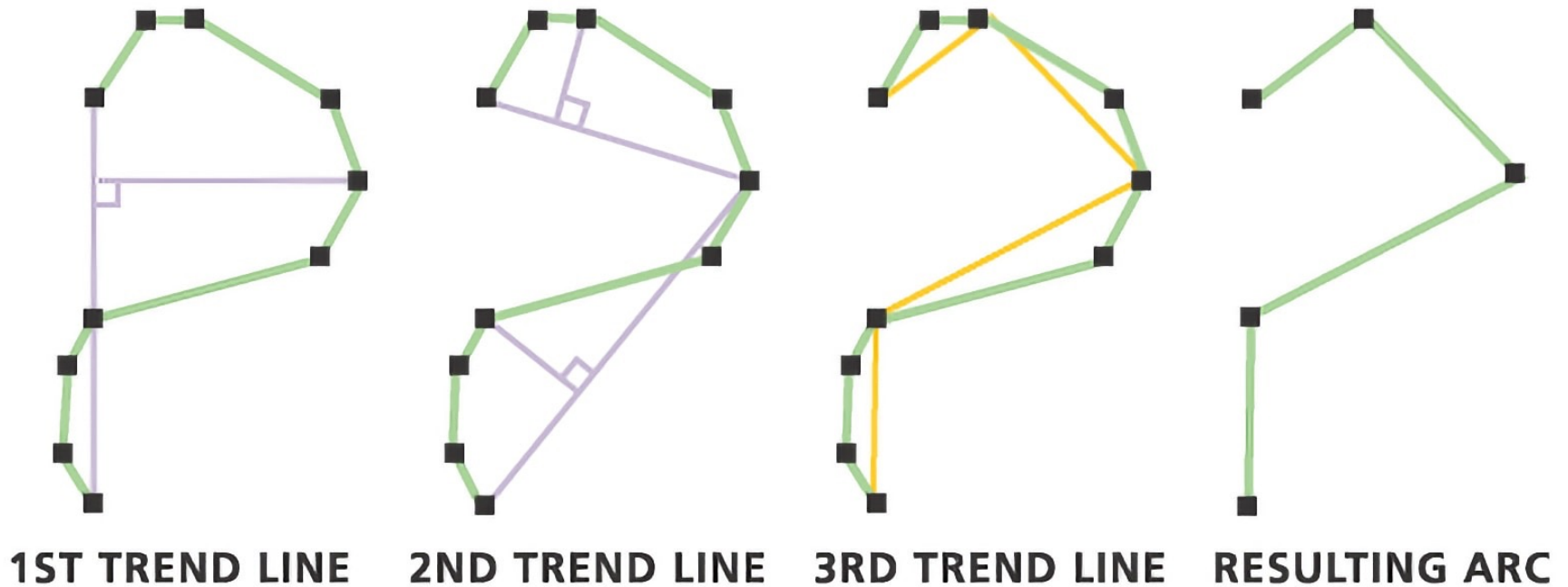
- Data Engineering
 - Build the system
 - Scalable pipeline design
 - Modular, reproducible code
 - Schema + storage strategy
 - Computational efficiency
 - Production-oriented thinking
- Industrial Engineering
 - Evaluate the decision impact
 - Sensitivity to spatial design choices
 - Analytics/risk interpretation
 - Distributional analysis of value
 - Operational implications
 - Policy and planning relevance
- Shared Expectation
 - Design and justify a defensible spatial workflow

Simplification



- Simplification is a process for **generalization** of vector objects (lines and polygons) usually for use in smaller scale maps.
- Another reason for simplifying objects is to **reduce** the amount of memory they consume (disk space and network bandwidth).
It may be wise to simplify complex geometries before publishing them as interactive maps :)

Douglas-Peucker algorithm

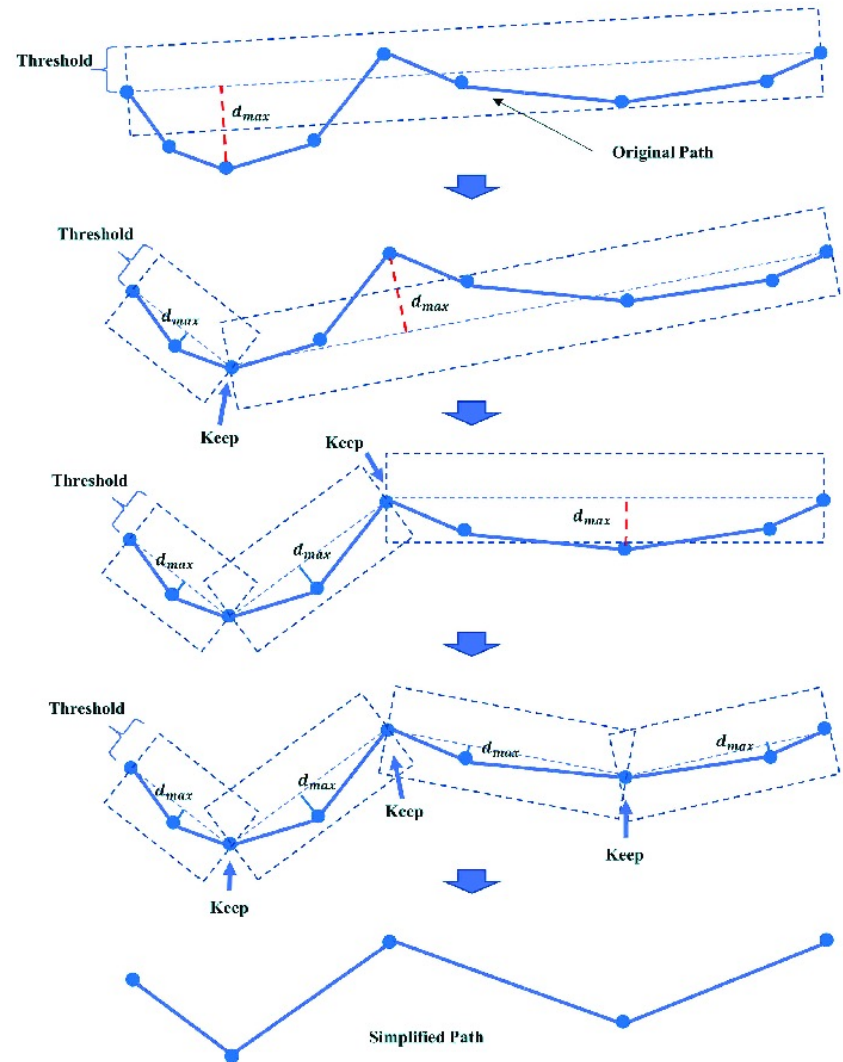


 **SIMPLIFICATION TOLERANCE**

Boundaries are simplified by **removing vertices**, decimating a curve composed of line segments to a *similar* curve with *fewer points*.

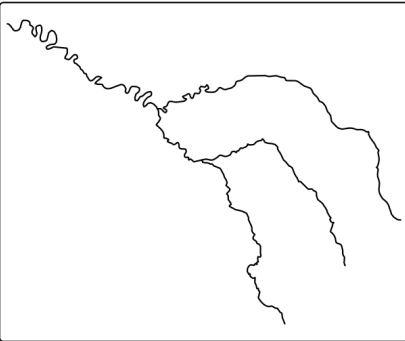
st_simplify()

- The sf package provides `st_simplify()`, which uses the **Douglas-Peucker algorithm** to reduce the vertex count.
- `st_simplify()` uses the **dTolerance** to control the level of generalization in map units.

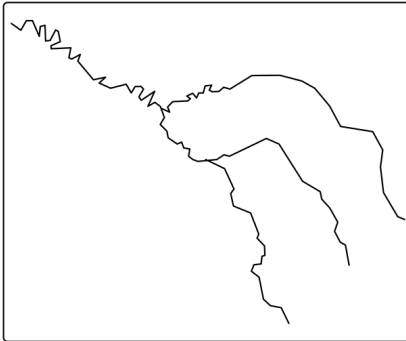


Simplification of a LINESTRING geometry representing the River Seine and tributaries

Original data



st_simplify



- The resulting object is a copy of the original but with fewer vertices.
- This is apparent, with the result being visually simpler and consuming less memory than the original object (as verified).

```
library(sf)
```

```
library(dplyr)
```

```
library(spData)
```

```
seine_simp = st_simplify(seine,  
  dTolerance = 2000) # 2000 m
```

```
object.size(seine)
```

```
# 18096 bytes
```

```
object.size(seine_simp)
```

```
# 9112 bytes
```

Simplification Example

Original data



st_simplify



```
us_states_simp1 =  
  st_simplify(us_states, dTolerance =  
    100000) # 100 km
```

- A limitation with **st_simplify()** is that it simplifies objects on a per-geometry basis
- This means the 'topology' is lost, resulting in overlapping and 'holey' areal units

Simplification Alternative

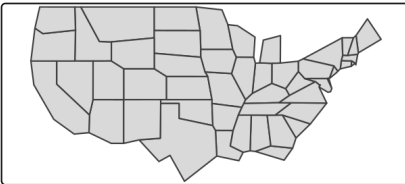
Original data



st_simplify



ms_simplify



- **rmapshaper** provides an alternative
- By default it uses the Visvalingam algorithm (which overcomes some limitations of the Douglas-Peucker algorithm)

```
us_states_simp1 =  
  st_simplify(us_states, dTolerance =  
    100000) # 100 km
```

```
# proportion of points to retain
```

```
# (0-1; default 0.05)
```

```
us_states_simp2 =  
  rmapshaper::ms_simplify(us_states,  
    keep = 0.01, keep_shapes = TRUE)
```

Simplification, Another Alternative

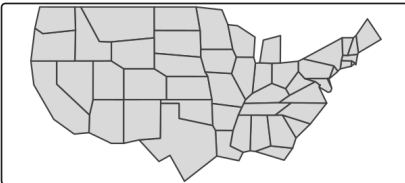
Original data



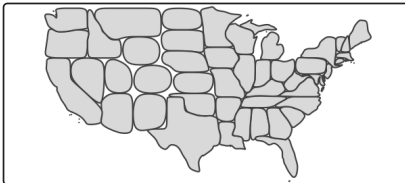
st_simplify



ms_simplify

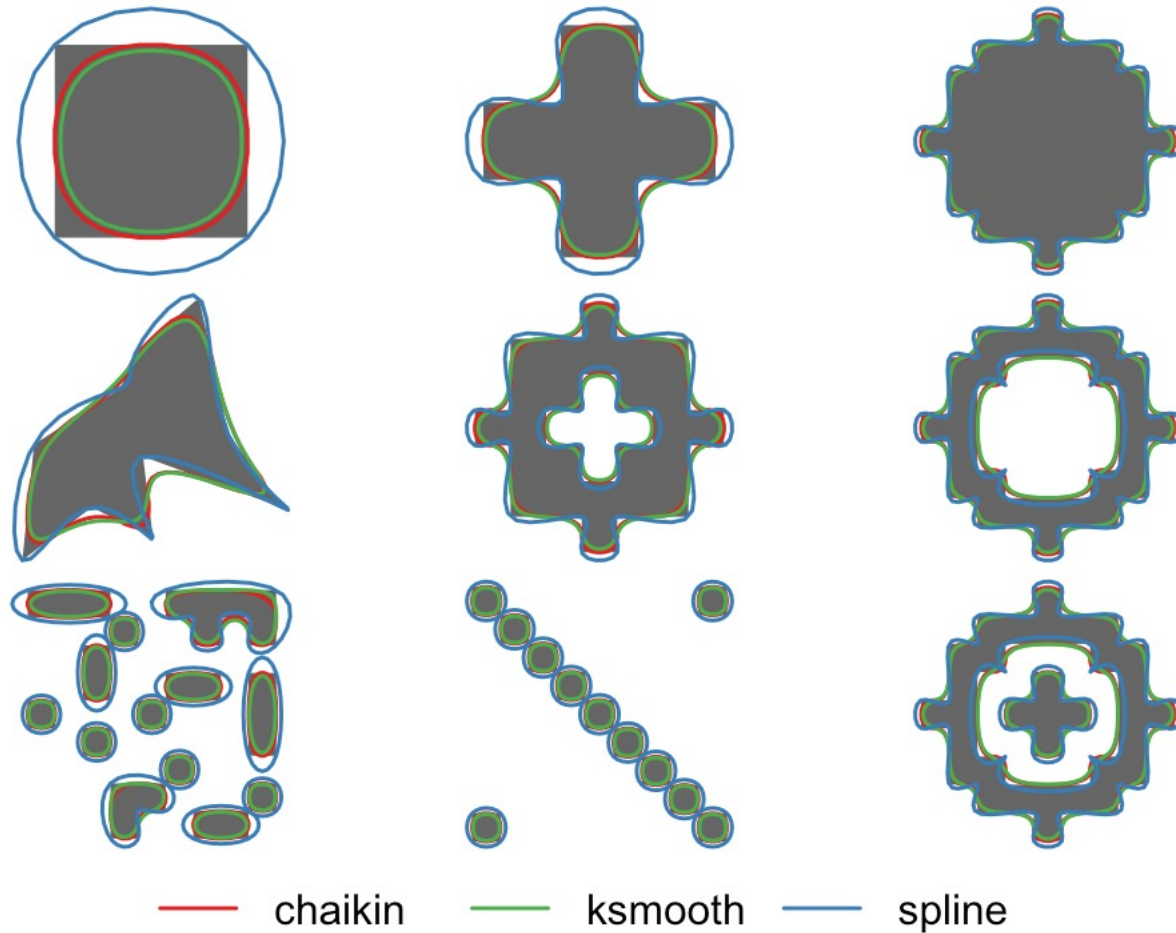


smooth(method = "ksmooth")



- **smoothr** implements three techniques for smoothing:
 - Gaussian kernel regression,
 - Chaikin's corner cutting algorithm, and
 - Spline interpolation
 - All described in the package website:
<https://strimas.com/smoothr/>
- Similar to **st_simplify()**, the smoothing algorithms don't preserve 'topology'

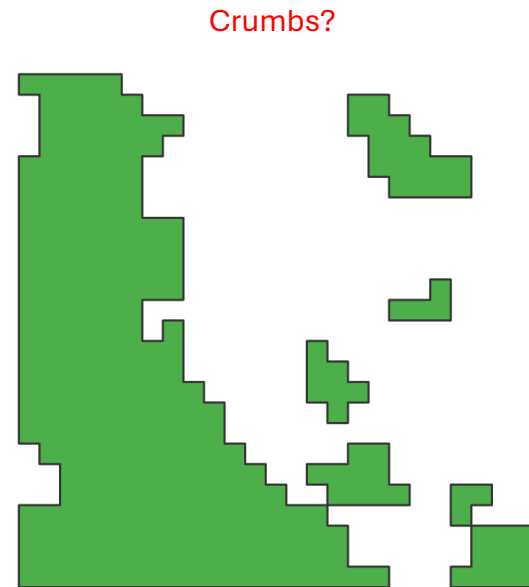
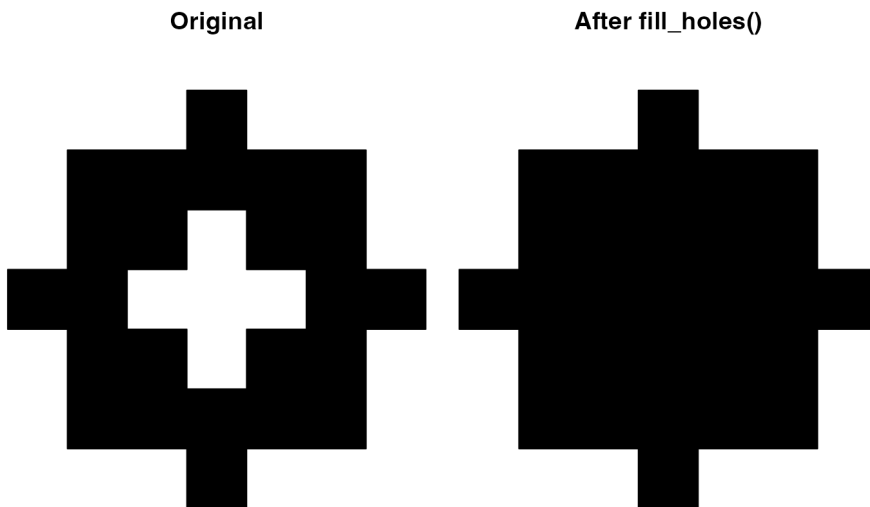
```
us_states_simp1 =  
  st_simplify(us_states, dTolerance =  
    100000) # 100 km  
  
# proportion of points to retain  
# (0-1; default 0.05)  
  
us_states_simp2 =  
  rmapshaper::ms_simplify(us_states, keep  
    = 0.01, keep_shapes = TRUE)  
  
us_states_simp3 =  
  smoothr::smooth(us_states,  
    method = "ksmooth", smoothness = 6)
```



smoothr offers a variety of tools for smoothing and tidying spatial features (i.e., polygons and lines) to make them more aesthetically pleasing, especially when converting raster data to vector format.

Also, on `smoothr`

- `drop_crumbs()` Removes small lines or polygons based on a length or area threshold
- `fill_holes()` Fill small holes in polygons



we will explore it more on raster-to-polygon conversion

Centroids

- Centroid operations identify the center of geographic objects
 - Like statistical measures of central tendency (including mean and median definitions of 'average'), there are many ways to define the geographic center of an object
 - All of them create **single point representations** of more complex vector objects
- The most used centroid is the **geographic centroid**



Geographic Center of the US

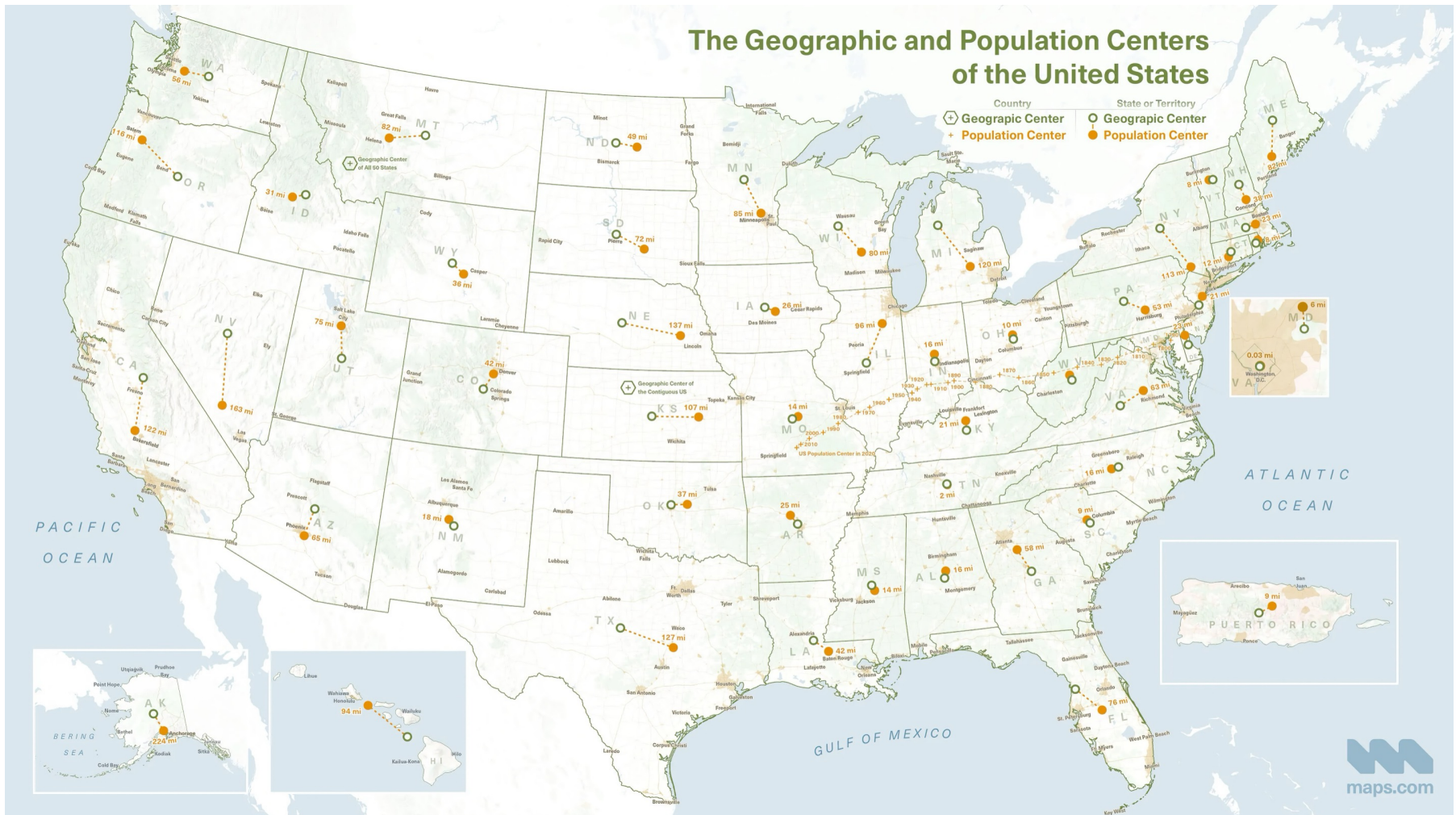


BELLEVILLE, GEOGRAPHICAL CENTER UNITED STATES AND CROSS ROADS OF AMERICA.

Centroids

- Geographic centroids have many uses, for example:
 - To create a simple point representation of complex geometries
 - To estimate distances between polygons
- Sometimes the geographic centroid falls outside the boundaries of their parent objects (think of a doughnut).
 - In such cases **point on surface** operations can be used to guarantee the point will be in the parent object (e.g., for labeling irregular multipolygon objects such as island states)

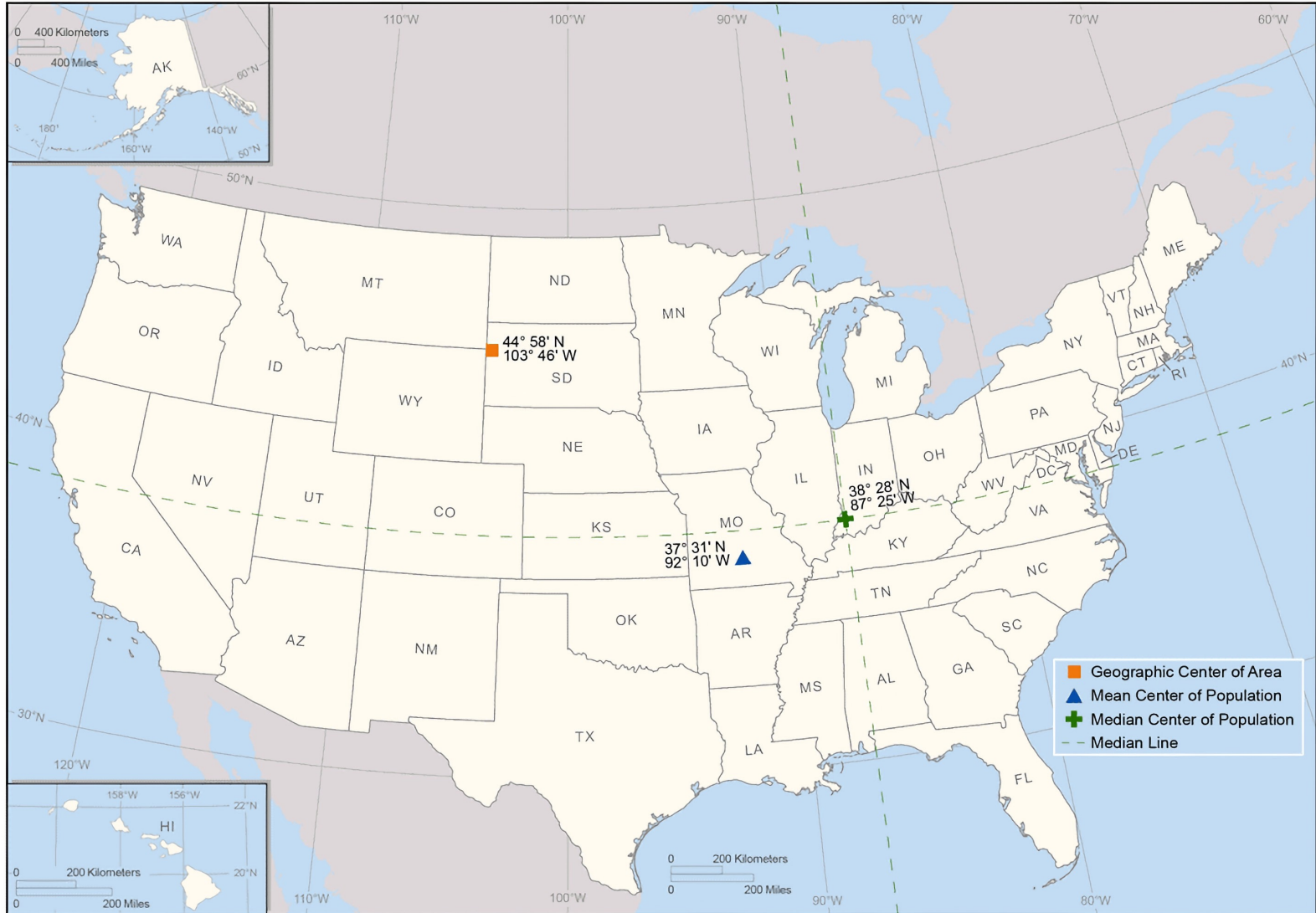
Geographic Centroids of Each State



<https://www.maps.com/mapping-the-middles-geographic-center-us/>

https://en.wikipedia.org/wiki/List_of_geographic_centers_of_the_United_States

Position of the Geographic Center of Area, Mean and Median Centers of Population: 2010



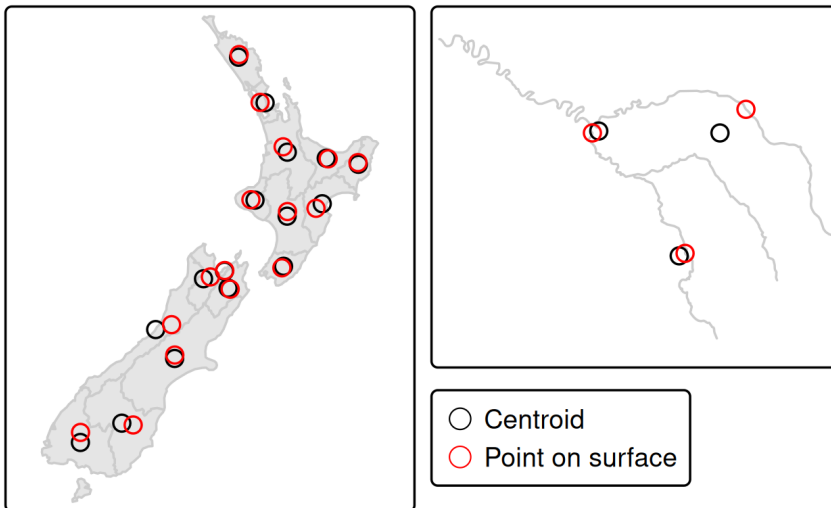
U.S. DEPARTMENT OF COMMERCE Economics and Statistics Administration U.S. Census Bureau

Prepared by the Geography Division

https://en.wikipedia.org/wiki/Geographic_center_of_the_United_States

Centroid Example

- Notice that the red points always lie on their parent objects!
- They are created with `st_point_on_surface()`



Warning message: `st_centroid/st_point_on_surface` assumes attributes are constant over geometries
>> The warning is reminding you that centroid/point on surface calculation is purely geometric; any associated attributes may not meaningfully reflect the point location.

```
nz_centroid = st_centroid(nz)
nz_pos = st_point_on_surface(nz)

plot(st_geometry(nz), col = "lightgray")
plot(st_geometry(nz_centroid), add = TRUE, pch = 19, col = "blue")
plot(st_geometry(nz_pos), add = TRUE, pch = 18, col = "red")

seine_centroid = st_centroid(seine)
seine_pos = st_point_on_surface(seine)

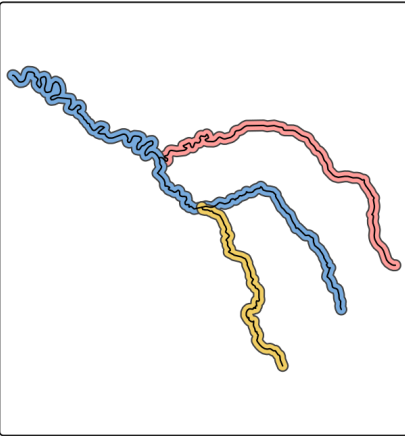
plot(st_geometry(seine), col = "black")
plot(st_geometry(seine_centroid), add = TRUE, pch = 19, col = "blue")
plot(st_geometry(seine_pos), add = TRUE, pch = 18, col = "red")
```

Buffers

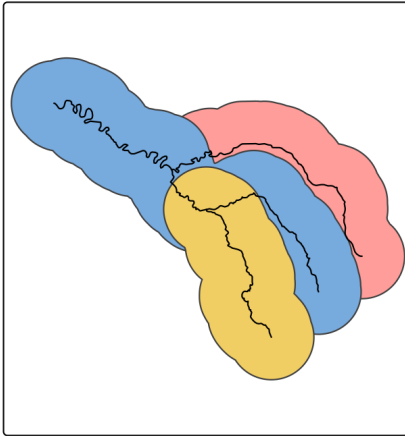
- Buffers are polygons representing the area within a given distance of a geometric feature
- Regardless of whether the input is a point, line or polygon, the output is a **polygon**
- Unlike simplification (which is often used for visualization and reducing file size), buffering tends to be used for geographic data analysis
 - How many points are within a given distance of this line?
 - Which demographic groups are within travel distance of this new shop?

Buffer Example

5 km buffer



50 km buffer



- Buffers of different sizes surrounding the River Seine and tributaries
- These buffers were created with command `st_buffer()`
- It requires at least two arguments: an input geometry and a distance, provided in the units of the CRS (in this case **meters**)

```
seine_buff_5km = st_buffer(seine,  
dist = 5000)
```

```
seine_buff_50km = st_buffer(seine,  
dist = 50000)
```

Negative Buffer (special case)

- Positive distances → expand outward from the feature
- Negative distances → shrink inward toward the feature interior (where valid)

Why Negative Buffers?

- Contract the geometry (e.g., to remove edge areas)
- Create inner rings inside polygons to represent an interior margin
- Model safety or exclusion zones inside a boundary

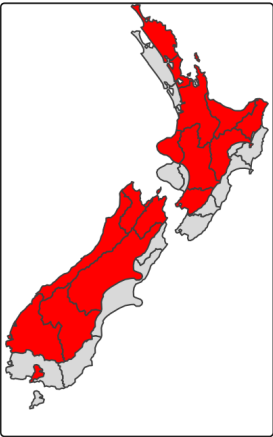
Affine Transformations

- Affine transformations are an essential part of geocomputation
- Affine transformation is any transformation that preserves lines and parallelism
 - However, angles or length are not necessarily preserved
- Affine transformations include, among others:
 - Shifting (translation)
 - Scaling
 - Rotation
- For example:
 - Shifting is needed for labels placement
 - Scaling is used in non-contiguous area cartograms

Shift Example

- Shifting moves every point by the same distance in map units
- It could be done by adding a numerical vector to a vector object

Shift



```
nz_sfc = st_geometry(nz)

# shift 100 km north (assuming
# projected CRS in meters)
nz_shift = nz_sfc + c(0, 100000)

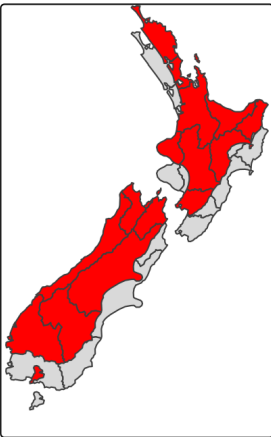
plot(st_geometry(nz_sfc), col =
     "lightgray")

plot(st_geometry(nz_shift), col =
     "red", add = TRUE)
```

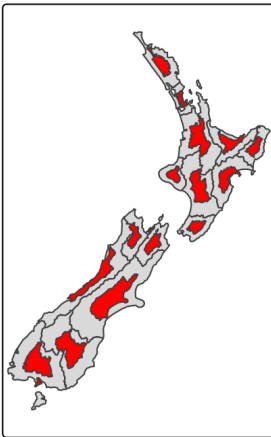
Scale Example

- Scaling enlarges or shrinks objects by a factor
- It can be applied either globally or locally
- Global scaling increases or decreases all coordinates values in relation to the origin coordinates, while keeping all geometries topological relations intact

Shift



Scale



```
nz_centroid_sfc = st_centroid(nz_sfc)
```

```
nz_scale = (nz_sfc - nz_centroid_sfc) *  
           0.5 + nz_centroid_sfc
```

```
plot(st_geometry(nz_sfc), col =  
     "lightgray")
```

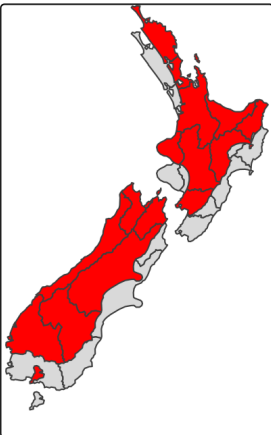
```
plot(st_geometry(nz_scale), col = "red",  
     add = TRUE)
```

Rotate Example

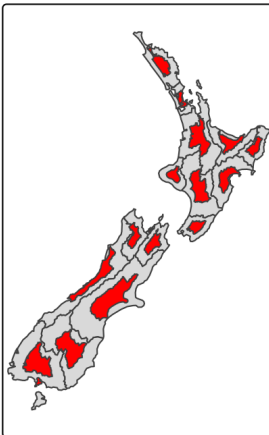
- The rotation function accepts one argument: a rotation angle in degrees
- Rotation could be done around selected points, such as centroids

```
rotation = function(a){  
  r = a * pi / 180 #degrees to radians  
  matrix(c(cos(r), sin(r), -sin(r),  
    cos(r)), nrow = 2, ncol = 2)  
}  
  
nz_rotate = (nz_sfc - nz_centroid_sfc)  
  * rotation(30) + nz_centroid_sfc  
  
plot(st_geometry(nz_sfc), col =  
  "lightgray")  
plot(st_geometry(nz_rotate), col =  
  "red", add = TRUE)
```

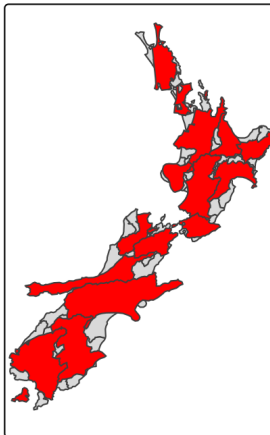
Shift



Scale



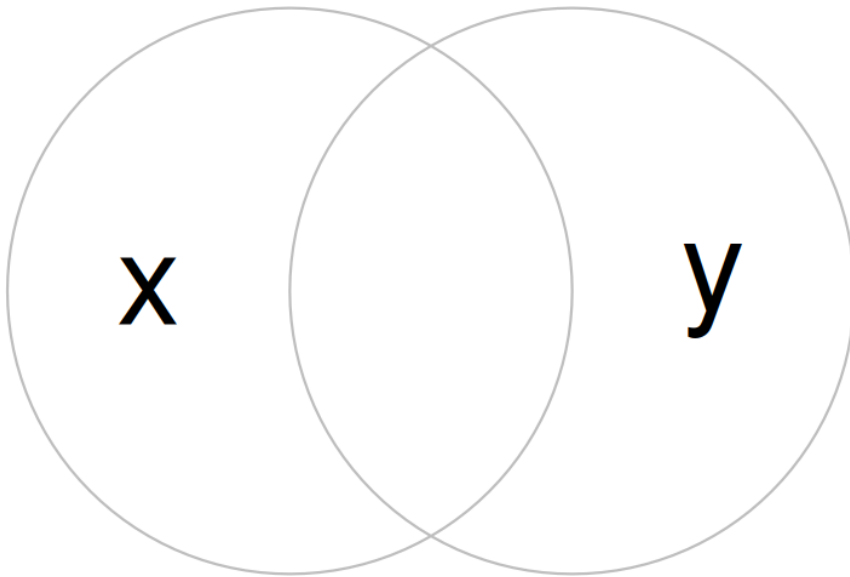
Rotate



Clipping

- Spatial clipping is a form of **spatial subsetting** that involves changes to the geometry columns of at least some of the affected features.
- Clipping can only apply to features more complex than points: lines, polygons and their 'multi' equivalents.

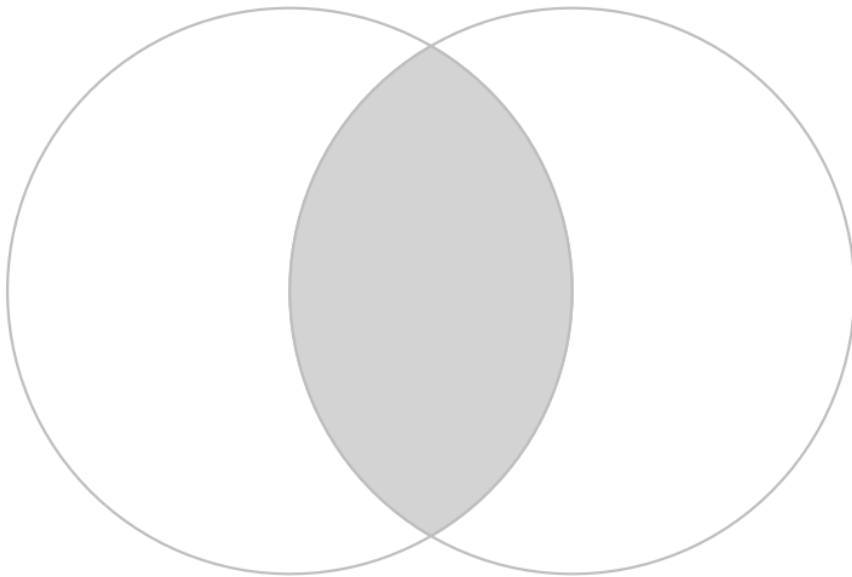
Clipping Example



- Imagine you want to select not one circle or the other, but the space covered by both x and y

```
b = st_sfc(st_point(c(0, 1)),  
           st_point(c(1, 1))) # create 2  
points  
  
b = st_buffer(b, dist = 1) #  
convert points to circles  
  
plot(b, border = "gray")  
  
text(x = c(-0.5, 1.5), y = 1,  
      labels = c("x", "y"), cex = 3) #  
add text
```

Clipping Example



- This can be done using the function `st_intersection()`

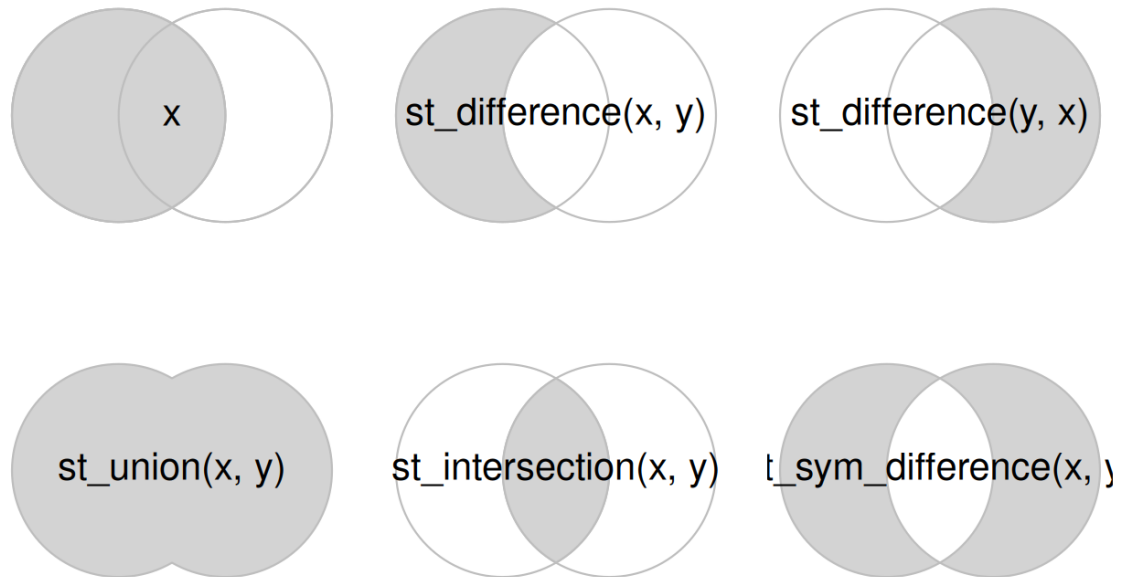
```
x = b[1]
```

```
y = b[2]
```

```
x_and_y = st_intersection(x, y)
```

```
plot(b, border = "gray")
```

```
plot(x_and_y, col = "lightgray",  
      border = "gray", add = TRUE) #  
      intersecting area
```



Spatial equivalents of logical operators

The subsequent code chunk demonstrates how this works for all combinations of the 'Venn' diagram representing x and y.

Subsetting and Clipping

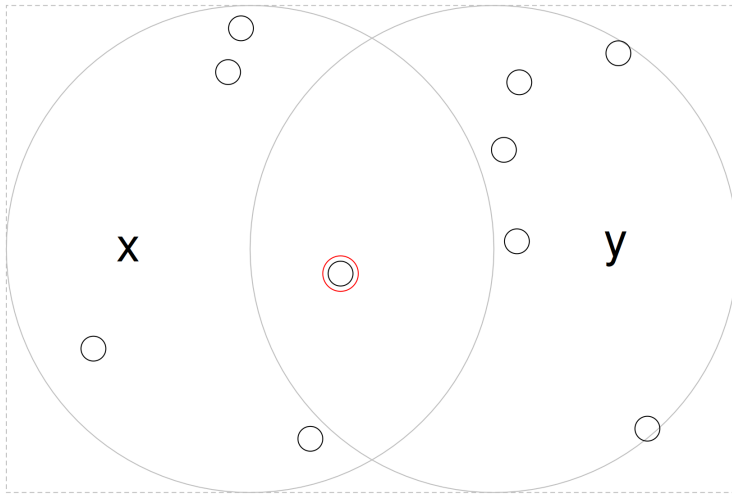
- Clipping objects can change their geometry
- But it can also subset objects, returning only features that intersect (or partly intersect) with a clipping/subsetting object

NOTE

- A bounding box is the smallest axis-aligned rectangle that fully contains a geometry
- Useful for: (i) quick summary of spatial extent, (ii) set plot limits or map viewports, (iii) crop or filter spatial data, (iv) align datasets before spatial joins, and (v) define spatial query windows
- **st_bbox()**
 - Defines the minimum and maximum coordinates that enclose ("**envelopes**") all geometries
 - Returns the bounding box of a spatial object
 - A named numeric vector with: xmin, ymin, xmax, ymax
 - Includes a CRS attribute
 - Can be coerced to: Numeric vector, Matrix, Polygon (**st_as_sf()**)

* https://postgis.net/docs/ST_Envelope.html

Subsetting Example



- Randomly distributed points within the bounding box enclosing circles x and y
- The point that intersects with both objects x and y is highlighted

```
bb = st_bbox(st_union(x, y))
```

```
box = st_as_sfc(bb)
```

```
set.seed(2024)
```

```
p = st_sample(x = box, size = 10)
```

```
p_xy1 = p[x_and_y]
```

```
plot(box, border = "gray", lty = 2)
```

```
plot(x, add = TRUE, border = "gray")
```

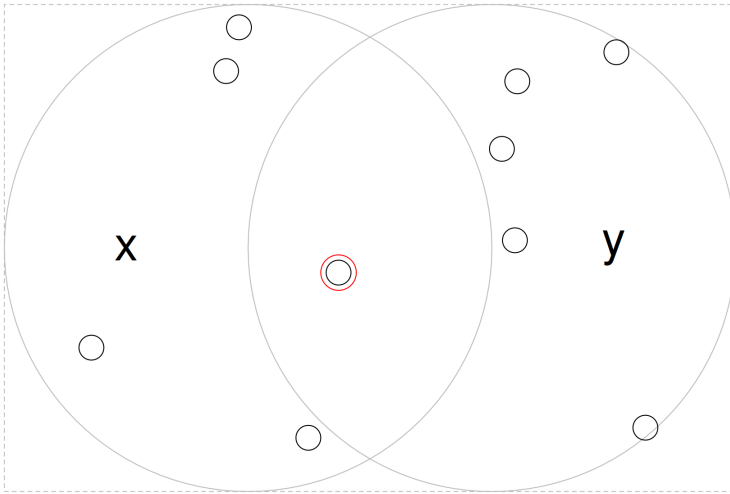
```
plot(y, add = TRUE, border = "gray")
```

```
plot(p, add = TRUE, cex = 3.5)
```

```
plot(p_xy1, cex = 5, col = "red", add = TRUE)
```

```
text(x = c(-0.5, 1.5), y = 1, labels =  
      c("x", "y"), cex = 3)
```

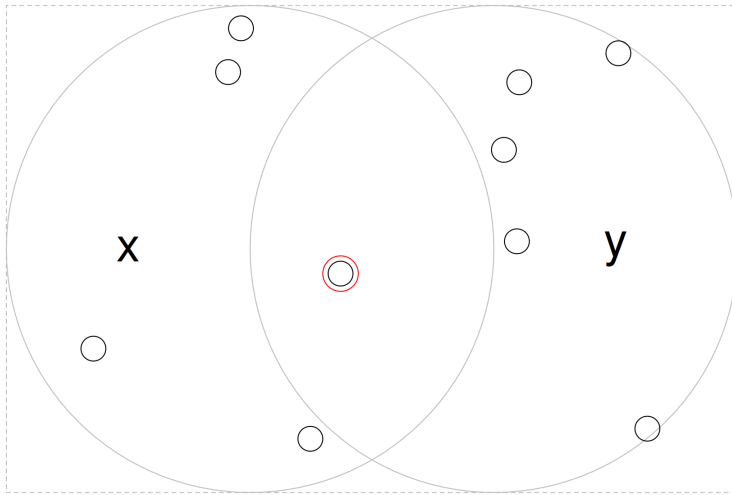
Subsetting Example



- Randomly distributed points within the bounding box enclosing circles x and y
- The point that intersects with both objects x and y is highlighted

```
bb = st_bbox(st_union(x, y))
box = st_as_sfc(bb)
set.seed(2024)
p = st_sample(x = box, size = 10)
x_and_y = st_intersection(x, y)
```

Subsetting Example



- The code chunk below demonstrates three ways to achieve the same result
- Which implementation to use?
 - Generally more concise implementations should be favored (1st approach)
 - We will return to the question of choosing between different implementations of the same technique or algorithm later

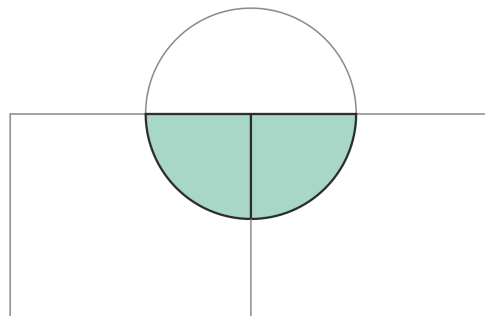
```
# way #1
p_xy1 = p[x_and_y]

# way #2
p_xy2 = st_intersection(p, x_and_y)

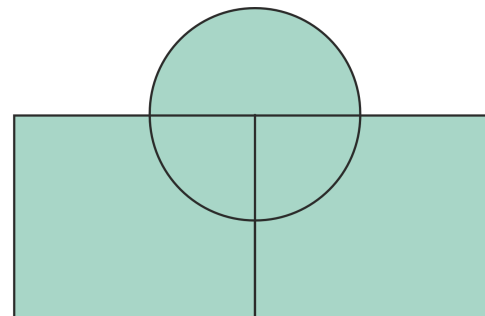
# way #3
sel_p_xy = st_intersects(p, x, sparse =
  FALSE)[, 1] & st_intersects(p, y, sparse
  = FALSE)[, 1]
p_xy3 = p[sel_p_xy]
```

Geometry Unions

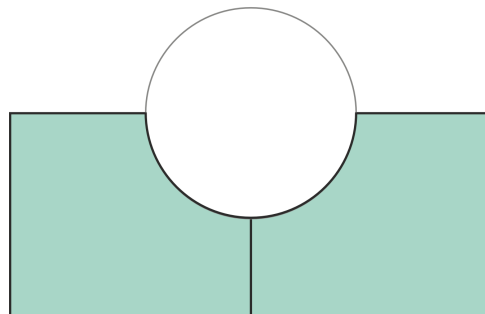
- Spatial aggregation can *silently* dissolve the geometries of touching polygons in the same group



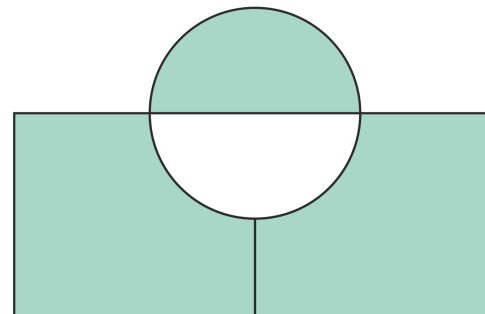
Intersection



Union



Difference

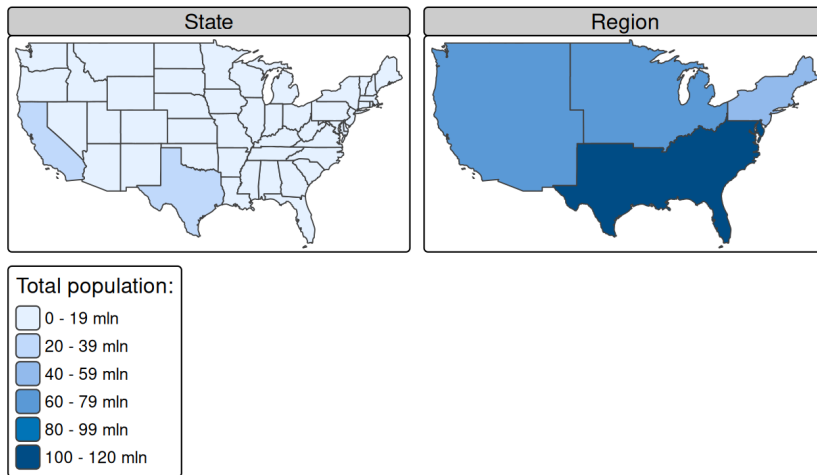


Symmetric difference

<https://pythongis.org/index.html>

<https://pythongis.org/part2/chapter-06/nb/08-overlay-analysis-with-vector-data.html>

Geometry Union Example



- Spatial aggregation on contiguous polygons, illustrated by aggregating the population of US states into regions, with population represented by color.
- The 48 US states and DC (`us_states`) are aggregated into four regions using base and `dplyr` functions
- Note the operation automatically dissolves boundaries between states

```
regions = aggregate(  
  x = us_states[, "total_pop_15"],  
  by = list(us_states$REGION),  
  FUN = sum, na.rm = TRUE)
```

```
regions2 = us_states |>  
  group_by(REGION) |>  
  summarize(pop = sum(total_pop_15,  
    na.rm = TRUE))
```

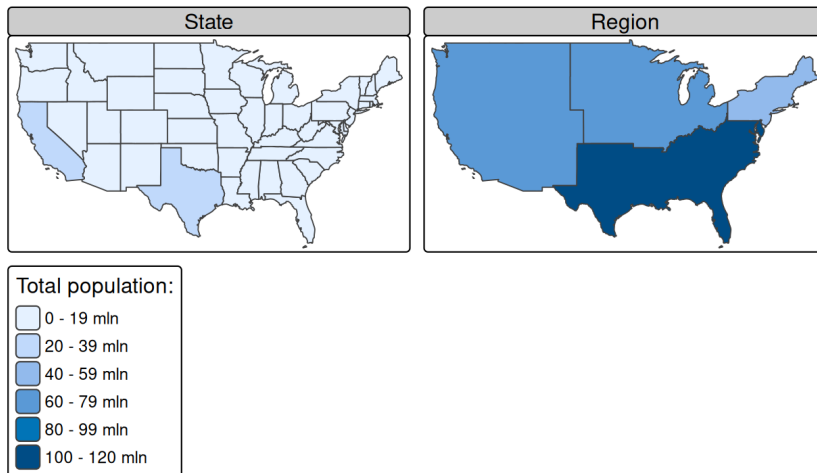
```
us_west = us_states[us_states$REGION ==  
  "West", ]
```

```
us_west_union = st_union(us_west)
```

```
texas = us_states[us_states$NAME ==  
  "Texas", ]
```

```
texas_union = st_union(us_west_union,  
  texas)
```

Geometry Union Example



- Behind the scenes, both **aggregate()** and **summarize()** combine the geometries and dissolve the boundaries between them using **st_union()**
- The function can also take two geometries and unite them, as demonstrated in the code chunk which creates a united western block incorporating Texas 😊

```
regions = aggregate(  
  x = us_states[, "total_pop_15"],  
  by = list(us_states$REGION),  
  FUN = sum, na.rm = TRUE)
```

```
regions2 = us_states |>  
  group_by(REGION) |>  
  summarize(pop = sum(total_pop_15,  
    na.rm = TRUE))
```

```
us_west = us_states[us_states$REGION ==  
  "West", ]
```

```
us_west_union = st_union(us_west)
```

```
texas = us_states[us_states$NAME ==  
  "Texas", ]
```

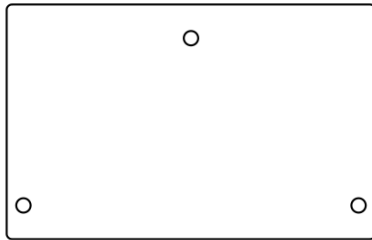
```
texas_union = st_union(us_west_union,  
  texas)
```

Type Transformations

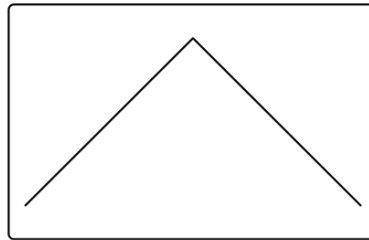
- **Geometry casting** is a powerful operation that enables transformation of the geometry type
- Conversion from multipoint to linestring is a common operation that creates a line object from ordered point observations, such as GPS measurements or geotagged media
 - This, in turn, allows us to perform spatial operations such as the calculation of the length of the path traveled
- Conversion from multipoint or linestring to polygon is often used to calculate an area
 - For example, from the set of GPS measurements taken around a lake or from the corners of a building lot
- Geometry casting of simple features geometry column (**sfc**) and simple features objects works the same as for **sfg** in most of the cases
 - One important difference is the conversion between multi-types to non-multi-types
 - As a result of this process, multi-objects of sfc or sf are split into many non-multi-objects

Type Transformations

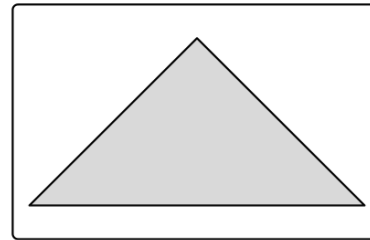
MULTIPOINT



LINESTRING



POLYGON



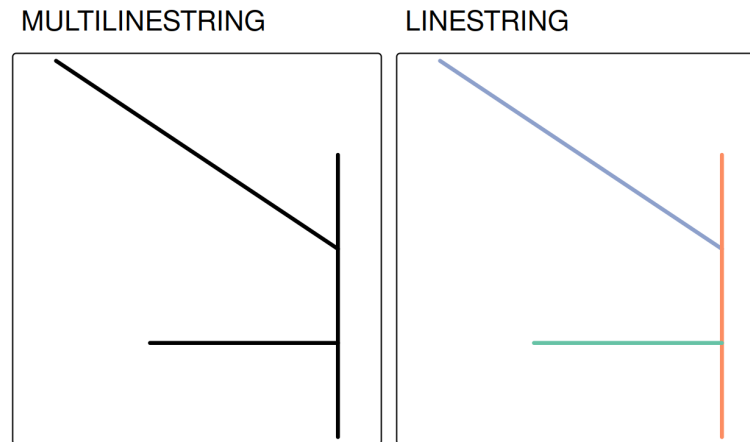
Examples of a linestring and a polygon casted from a multipoint geometry

```
multipoint = st_multipoint(matrix(c(1, 3, 5, 1, 3, 1), ncol = 2))
```

```
linestring = st_cast(multipoint, "LINESTRING")
```

```
polyg = st_cast(multipoint, "POLYGON")
```

Type Transformations



Examples of type casting between multilinestring (left) and linestring (right)

```
multipoint_2 = st_cast(linestring, "MULTIPOINT")  
multipoint_3 = st_cast(polyg, "MULTIPOINT")
```

```
all.equal(multipoint, multipoint_2)  
# [1] TRUE  
all.equal(multipoint, multipoint_3)  
# [1] TRUE
```

Assignment #3

Due Mon Feb 23 at 5 PM

Background

- You are asked to:

Integrate parcel data, FEMA floodplain data, and US Census geographic boundaries to evaluate flood risk exposure of single-family structures and its relationship to parcel improvement value in Brazos County, TX

- You must clearly document:

1. Your data acquisition strategy
2. Your data cleaning and preprocessing steps
3. Your spatial design decisions (projection, geometry transformations, buffering logic, joins)
4. Your reasoning for analytical choices

- Use the tools we learned so far >> reproducibility and clarity are essential!

Introductions

- This assignment is *intentionally* designed with limited procedural instructions
One of the primary objectives is for you to demonstrate how you plan, structure, and justify a spatial data engineering workflow
- Grade will consider as much on analytical design decisions as on final outputs
- Same data. Same base assignment. Different focus.
 - DAEN: Build the infrastructure for spatial analytics
 - ISEN: Evaluate the decision of spatial analytics
 - Shared Expectation: Design and justify a defensible spatial workflow
- As you work on the assignment, please be prepared to explain and walk through your decisions if asked!
 - You should be able to clearly describe what you did and why
 - Make sure you truly understand your workflow and results

Part 1: Data Acquisition and Planning

1. Obtain Brazos County Parcel Data <https://brazoscad.org/tax-information/gis/>

- Ensure the dataset includes:
 - Parcel geometry
 - Improvement value (or comparable building value metric)
 - Parcel identifier
- Document:
 - Data source and format
 - Coordinate reference system (CRS)
 - Any missing or inconsistent fields

2. Transform Parcels to Points

- Convert parcel polygons to point representations
- Justify:
 - Whether you use centroids, interior points, or another approach
 - The implications of this choice for flood risk classification
 - Potential bias introduced by large or irregular parcels
- Discuss:
 - What happens if a parcel intersects multiple flood zones?
 - Does a point representation simplify or distort risk classification?

Part 2: Flood Risk Engineering

1. Acquire FEMA flood hazard layer(s) for Brazos County

<https://floodmaps.fema.gov/NFHL/status.shtml>

- Document :
 - Dataset details, CRS, geometry type
 - Flood zone classification fields

2. Reclassify Floodplain Categories

- **Reclassify flood zones into 100-year, 500-year, or Outside floodplain**
- Define:
 - Which FEMA zone codes correspond to each category
 - How ambiguous or special zones are handled
 - Provide a reclassification table in your report

3. Assign a flood risk category to each parcel (*spatial join?*)

- Should parcels be intersected as polygons?
- Should parcels be converted to points first?
- Does buffering make sense? If so, what should be buffered? Parcels or floodplain?
- Why? Explain your approach

Part 3: Census Geography Integration

1. Acquire county boundaries from the U.S. Census Bureau (e.g., [TIGER/Line](#) or [tigris](#) or [tidycensus](#))
 - Check Brazos County geometry
 - Ensure CRS compatibility with other layers
2. Acquire Census "Places" boundaries
 - Document:
 - Definition of "place"
 - How incorporated and census-designated places differ (if relevant)
3. Assign Parcels to Places
 - Using a spatial join:
 - Assign each parcel to a Census Place
 - Then filter to include only Places located in Brazos County
 - Discuss:
 - How you handle parcels that fall outside any Place
 - Whether to rely on intersection or centroid containment

Part 4: Summary Statistics

- After constructing the final parcel dataset with:
 - Flood risk classification + Place assignment + Improvement value
- Produce the following summary statistics:
 - **4.1 Parcel Counts**
 - Total number of single-family parcels in:
 - 100-year floodplain, 500-year floodplain, Outside floodplain
 - In-risk vs out-of-risk totals (define clearly what qualifies as "at risk")
 - **4.2 Improvement Value Comparisons**
 - Compute average improvement value for parcels:
 - Inside flood risk areas and outside flood risk areas
 - Separate averages for 100-year vs 500-year zones
 - Median values and standard deviation
 - **4.3 Possible questions to discuss:**
 - Are differences economically meaningful?
 - Are there extreme values influencing the mean?
 - Would transformation (e.g., log values) be appropriate?
 - Also:
 - Any tradeoffs between computational efficiency and spatial precision.
 - Potential sources of misclassification, or assumptions that could affect conclusions.

Deliverables

Submit:

1. A reproducible script or notebook (R or Python)
2. A short technical report explaining:
 - Data sources
 - Workflow design
 - Key assumptions
 - Justification for spatial decisions
 - Interpretation / summary results (*i.e., the tables below*)(4–5 pages recommended)
3. Tables of summary statistics
4. Final engineered dataset (parcels with flood and place attributes)

Question to Practice

1. Why is selecting an appropriate projected CRS important when performing buffering or area-based spatial operations?
2. When assigning flood risk to parcels, what is the primary tradeoff between using parcel centroids versus full parcel polygons?

Discuss:

3. Under what circumstances would buffering floodplain polygons make sense? Data misalignment? Vertical vs horizontal uncertainty? Policy-driven safety margins? When might buffering introduce bias?
4. When reclassifying detailed FEMA flood zones into 100-year and 500-year categories: What information is lost? When is aggregation justified? Could collapsing categories hide meaningful differences in risk?
5. Suppose average improvement value is higher outside flood risk areas. Does this imply flood risk reduces property value? What confounding factors might exist (e.g., land use, zoning, development age)? Would median be more appropriate than mean? Why?